

TableClaw TODO

用途：维护 TableClaw 的迭代计划与待办。完成项打勾不删，保留迭代轨迹。

分工说明：

- 本文件只记 要做什么 / 在做什么 / 做完打勾。
- 历史细节、决策原因、踩过的坑写到 `development-log.md`。
- 完成一项时：本文件改 `- [ ]` → `- [x]`，并在 dev-log 里补一段说明，互相引用。

最后更新：2026-05-29

项目定位（不要忘记）

TableClaw 是表格专精 agent，QA 只是其中之一。商用形态需要覆盖至少 4 类场景：

场景类	例子
QA / 分析	读表、按指标查询、筛选、排名、统计
编辑 / 修复 / 清洗	填公式、补数据、清重复行、修复类型、统一格式
表格转换 / 跨表合并	CSV ↔ xlsx、多 sheet 拆合、纵转横、跨表 vlookup、多期合并
报表生成 / 从 0 建表	从多张表生成 dashboard / KPI 报表、含图表与条件格式；从零搭新业务表模型

设计每个 skill / tool / 评测题时，反复对照这 4 类，避免又退化回"只服务 QA"。

近 7 天进度

每周一扫一眼。 已完成、 在做、 计划中。

-  整理目录结构与文档（mentor demo 路线收尾、归档、.gitignore）
-  修正项目定位 → 表格专精 agent，覆盖 QA + 编辑 + 转换 + 报表 4 大场景
-  建立 TODO + dev-log 双文档分工
-  git init + 修评分器 + 写 TableClaw native xlsx skill + 跑对照评测
-  启动 dataset 扩充（招募同学）+ tool RFC 起草 + 多家 skill 横评准备

近期：本周完成（基础设施 + native skill 第一版）

工程基线

- ☐ **git init + 第一次 commit**: 把当前整理后的状态作为基线。 `.gitignore` 已就绪。
- ☐ **修评分器 (最小改动)**: `run_eval.py:_fact_matches` 里 `name=value` (value 是数字) 改走 numeric tolerance; 保留现有 evaluation 字段不变。fix 完用现有 10 任务跑一遍验证 auto pass 数字真实反映正确率。

Native Skill v0: 先占位 4 大场景，等 mentor 对齐后展开

- ☐ **codex 原文备份到 _archive/**: `nanobot/nanobot/skills/_archive/codex-spreadsheets-original.md`, 保留 1068 行原文不丢。
- ☐ **重写 nanobot/nanobot/skills/xlsx/SKILL.md**: ≤200 行的 TableClaw native skill v0。
 - 描述里明确声明 4 大场景全覆盖 (QA / 编辑 / 转换 / 报表)。
 - 内容上吸收三家强项中适合表格全场景的部分:
 - anthropic: openpyxl + `data_only=True` + 全精度 + LibreOffice 重算 + 公式错误扫描
 - kimi: inspect → recheck → reference-check → validate workflow
 - codex: verify pass、render-before-deliver、artifact 质量标准 (不依赖 artifact-tool)
 - 不写死场景顺序——v0 是入口 skill, 后续按需拆子 skill。
- ☐ **跑 native skill vs codex 对照评测**: `./eval.sh` 全量 10 任务 × skill-on/off。产出 `docs/实验评测/skill-matrix/` 新报告, 用修过的评分器。
- ☐ **mentor 同步**: 拿对照评测数据 + 4 大场景定位陈述去对齐, 确定后续主推哪个场景。

中期: 1-2 周 (覆盖 4 大场景的实验体系)

Eval Dataset 规模化 (核心阻塞项)

目标: 从现在 11 题 (10 主线 + 1 demo) 扩到 **80-150 题学术 benchmark 级**, 覆盖 4 大场景 × 三档难度, 能发 paper / 写技术报告 / 给客户看说服力。后续所有 skill 与 tool 的边际效果证明都依赖这套 dataset。

规模分配 (初拟, 可调):

场景	题数	simple	medium	hard
QA / 分析	30-40	10-13	10-13	10-14
编辑 / 修复 / 清洗	20-30	7-10	7-10	6-10
表格转换 / 跨表合并	15-25	5-8	5-8	5-9

报表生成 / 从 0 建表	15-25	5-8	5-8	5-9
合计	80-120			

难度二维（运算复杂度 × 表结构复杂度）：

- 运算复杂度：simple（单步查询） / medium（双步骤组合） / hard（多步推理、跨期、聚合 + 排序）
- 表结构复杂度：flat（单 sheet、单层表头） / nested（多级表头、merged cell） / multi（多 sheet、跨表）
- 任务 `evaluation.complexity` 同时打两个维度的标签，方便后续做矩阵分析

数据来源（混合策略）：

- ☐ 来源 A — 现有 `test_table/`：四川经营数据系列，估 20-30 题（多样性弱但 gold 容易核实）
- ☐ 来源 B — 公开 **spreadsheet benchmark**：评估并部分引入
 - SpreadsheetBench（GitHub microsoft/...）
 - WikiTableQuestions
 - TabFact
 - FeTaQA
 - 估 30-50 题，注意 license 兼容
- ☐ 来源 C — 同学/朋友贡献真实业务表：覆盖更多行业（财务/HR/营销/运营），估 20-30 题
 - 隐私清洗：删公司名、人名、ID 等敏感字段
 - 给同学发标准 jsonl 模板 + README，统一字段
- ☐ 来源 D — 合成（兜底）：从公开数据集生成边界 case（极端值、空行、重复行、超大表），估 5-10 题

协作方式（要找同学帮忙）：

- ☐ 建一个 `eval_test/test_dataset/contributions/` 子目录，每人一个 jsonl 文件
- ☐ 写 `eval_test/test_dataset/CONTRIBUTING.md`：jsonl 字段说明、gold answer 核实流程、提交方式
- ☐ 一题一 GitHub issue 模板（如果项目托管），用 label 区分场景/难度/状态
- ☐ 每周一次 batch review，合入主 dataset

Gold answer 流程：

- ☐ 模型先答 + 出题者人工核验
- ☐ hard 题至少二人交叉核验
- ☐ 数值用 tolerance（1e-6 或题目自定），文本用 fact match + LLM-as-judge 兜底（可选）

Checkbox（前置决策完成后逐步展开）：

- ☐ 写 `eval_test/test_dataset/CONTRIBUTING.md` 投稿规范
- ☐ 写 `eval_test/test_dataset/SCHEMA.md` 任务 jsonl 字段标准 (含 complexity 二维标签、4 场景枚举)
- ☐ 评估 5 个公开 benchmark 的可引入性 + license, 挑 2-3 个
- ☐ 招募 3-5 名同学, 每人认领 10-20 题
- ☐ 写公开 benchmark 引入脚本 (格式转换到 TableClaw jsonl schema)
- ☐ 扩 `run_eval.py` 支持非 QA 任务的判分:
 - ☐ 输出文件 sha 对比
 - ☐ 输出 cell 值 / 公式正确性比对
 - ☐ 行数列数维度比对
 - ☐ 公式错误扫描 (#REF!/#DIV/0! 等)
- ☐ 跑通 baseline: 在新 dataset 上跑当前 native skill v0, 建立基线分

表格专用 Tool 系列 (动 nanobot 本体, 已许可)

目标: 把模型反复在 exec 里写的 openpyxl 套路沉淀成 nanobot tools, 让模型一次调用就拿结果, 减少试错、降 token、稳定性翻倍。接入位置:

`nanobot/nanobot/agent/tools/tableclaw/<name>.py` (建子目录避免污染上游 tool 文件夹)。

命名规范:

- 全部 `tableclaw_<动词>` 或 `tableclaw_<动词>_<对象>` 形式, snake_case
- 动词在前, 对象明确, 不重复 `table` 前缀
- 入参第一位永远是 `path` (表文件路径), 其余按重要性排序
- 返回 dict 或 dataclass, 包含 `data` + `meta` (含执行 sha、耗时、warnings)
- 失败抛 `TableclawError` 含详细原因, 不让模型猜

第一批 5 个 tool (按依赖顺序):

- ☐ `tableclaw_inspect(path, sheet=None)` — 返回 sheet 列表 / rows / cols / merge 范围 / 多级表头 / total 行 / blank 行
- ☐ `tableclaw_locate_column(path, period, group, metric, sheet=None)` — 三元组定位列, 处理 merged header; 返回 column index + 上下文片段
- ☐ `tableclaw_aggregate(path, value_col, group_by_col, agg='sum', sheet=None, row_filter=None)` — 按某列分组聚合, 支持 sum/mean/max/min/count
- ☐ `tableclaw_filter(path, conditions, sheet=None)` — 阈值/范围/跨期 conjunction 过滤, conditions 是 list of {col, op, value}
- ☐ `tableclaw_topk(path, value_col, k=10, ascending=False, sheet=None, ...)` — Top K 排名, 支持多列聚合与多列排名

`row_filter=None)` — Top/Bottom-K 排名, 含 tie 处理与多列 tie-breaker

Tool RFC (先文档后代码) :

- ☐ 写 `docs/功能开发/tableclaw-tools-rfc.md`: 每个 tool 的签名、入参、返回、错误码、典型用例。先 RFC 评审再写代码。
- ☐ 在 RFC 文档里和 nanobot `agent/tools/base.py` 的接口规范对齐

验证三道关 (每个 tool 都要过) :

- ☐ **单元测试**: `nanobot/nanobot/agent/tools/tableclaw/tests/test_<name>.py`, 覆盖:
 - 有/无 merge
 - 多 sheet / 单 sheet
 - 边界数据 (空表、单行、全 None 列)
 - 类型异常 (字符串混入数字列)
 - 大文件 (300+ 列、200+ 行)
- ☐ **集成测试**: 在现有 dataset (先 11 题, 扩充后再跑全量) 上跑, 记录每个 tool 的:
 - 触发次数
 - 节省的 exec 步数
 - token 降幅
- ☐ **A/B 对比**: 让模型 prompt 里看到 tool vs 看到等价的 SKILL.md, 跑对照。要看到 tool 形态稳定胜出才合格。

第一批跑通后再扩 (候选, 按需) :

- `tableclaw_diff` — 跨期/跨表差异, 输出 added/removed/changed
- `tableclaw_pivot` — 纵转横、宽表 ↔ 长表
- `tableclaw_write_cell` / `tableclaw_write_formula` — 写入操作
- `tableclaw_validate` — 公式错误扫描
- `tableclaw_render_preview` — 渲染成图/HTML 供 verify pass

多家 Skill 横评 (决定 TableClaw skill 路线)

目标: 诚实测出 codex / kimi / anthropic / claude-code / TableClaw 自研在**同一任务集**上的边际效果, 决定 skill 路线 (继续优化自研、还是基于某家底改) 。

参评 skill (带依赖跑, 真实路径) :

- ☐ codex spreadsheets skill — 现有 `skills/codex/SKILL.md` 1068 行。需要: node + `@oai/artifact-tool`
- ☐ kimi xlsx skill — 现有 `skills/kimi_xlsx_skill/`。需要: python + pandas + openpyxl + `KimiXlsx` Linux ELF 二进制 → **docker** 容器隔离跑

- ☐ anthropic xlsx skill — 现有 `skills/anthropic_xlsx_skill/`。需要：python + openpyxl + pandas + LibreOffice + `scripts/recalc.py` + `scripts/office/` helpers（之前裁剪过，要补回）
- ☐ claude-code 自带 spreadsheet skill — 需要先弄到原文（从 claude.ai 安装包或在线版抓取）
- ☐ TableClaw 自研 v0 — 当前 `nanobot/nanobot/skills/xlsx/SKILL.md`（rewrite 后）
- ☐（可选）copilot for excel 风格 skill — 如果能搞到 prompt 原文

依赖隔离方案：

- ☐ 每家做一份 `docker-compose.<vendor>.yaml`，跑 multi-skill harness 时拉对应 image
- ☐ kimi 的 `kimiXlsx` 路径写死 `/app/.kimi/...` 在 docker 里可以模拟
- ☐ anthropic 的 `scripts/office/` helper 从原仓库或社区找回，docker image 装 LibreOffice
- ☐ 不可用时**明确报失败，不降级**，避免数据失真

Harness 改造：

- ☐ `eval_test/run_multi_skill.py`：接受 skill 集合参数，对每家分别跑一遍同一 dataset
- ☐ 配置形式：`nanobot/configs/tableclaw-multiskill-<vendor>.json`，每家一份 disabledSkills + 依赖检查
- ☐ 输出：`eval_test/results/multi_skill/<vendor>/run.json`，统一格式方便横比
- ☐ 报告生成器：`docs/实验评测/multi-skill/comparison.md`，矩阵表（vendor × 场景 × 难度 × 指标）

评测维度：

- token（total / prompt / completion / cached）
- 正确率（auto pass + 人工核验）
- skill 读取率（这家 skill 真的被模型读到了几次）
- 步数（tool steps）
- 答案精度（数值小数位、文本完整性）
- **鲁棒性**：每题跑 N 次（建议 N=3 或 5），取分布而不是单次结果

Checkbox：

- ☐ 写 RFC：`docs/功能开发/multi-skill-eval-rfc.md`，定 harness 接口、依赖隔离、指标定义
- ☐ 给每家做依赖 dockerfile（5 份）
- ☐ claude-code skill 原文获取（待你确认渠道）
- ☐ `run_multi_skill.py` 实现 + multi-seed 跑
- ☐ 横评报告（先在 11 题上跑通，dataset 扩到 80+ 后再跑大版本）

- ☐ 写 docs/实验评测/multi-skill/decision.md：基于横评结果决定 TableClaw skill 路线（自研继续优化 / 基于某家底改 / 多家组合）

Skill 拆分（等 mentor 对齐后启动）

- ☐ 依据 mentor 反馈 + 横评结果，把 v0 入口 skill 拆成多个子 skill，可能形态：
 - ☐ tableclaw-table-qa
 - ☐ tableclaw-table-edit
 - ☐ tableclaw-table-transform
 - ☐ tableclaw-table-report
- ☐ 建立 skill 选择实验：每个子 skill 单独 disable 测试边际贡献。

长期：1-2 个月（产品原型走向商用）

模型 / 部署

- ☐ 多模型适配：除 dashscope/deepseek 外至少跑通：
 - ☐ anthropic 一份 config (claude-sonnet/opus)
 - ☐ openai 一份 config (gpt-4 / gpt-4o)
- ☐ 本地小模型可选项（是否做待 mentor 决定）：是否要在 ollama / llama.cpp 上跑通一份，覆盖隐私敏感场景。

对外接口

- ☐ OpenAI 兼容 HTTP API 启用：nanobot 自带 /v1/chat/completions，需要测一遍 + 写示例调用文档。
- ☐ WebUI 演示页：选其一
 - ☐ 启用 nanobot 自带 webui (gateway)
 - ☐ 独立小页面读 eval_test/results/ 做 timeline + token 可视化

安全 / 上线

- ☐ 清掉所有硬编码 API key：start.sh / eval.sh / demo.sh / run_eval.py 里的 sk-... 默认值改成必须从 env / .env 读取。
- ☐ 配置鉴权：对外 API 加最简 token / IP 白名单。
- ☐ 打包发布：决定是 docker image 还是 pip 包，写 README + 一键部署脚本。

文档化

- ❑ 写 **TableClaw README.md** (项目根, 目前只有 nanobot 的 README) : 定位、4 大场景、quick start、架构图。
- ❑ 重画 **skill pipeline svg**: 之前删过的 `tableclaw-skill-pipeline.svg`、`tableclaw-xlsx-case-flow.svg` 后续要补回, 对应商用版能力。

已完成

完成项移到这里, 不删。日期格式 `YYYY-MM-DD`。

2026-05-28

- ✓ 跑通 nanobot + 百炼 dashscope 配置 — 见 dev-log
- ✓ 一键启动脚本 `start.sh`
- ✓ workspace 迁到项目内 `workspace/`
- ✓ 搭第一版 `eval_test/test_dataset/` (2 任务 smoke)
- ✓ 接 codex xlsx skill 作为最小可演示 skill 选择机制
- ✓ Token usage 运行时持久化 (`workspace/usage/usage.jsonl`)

2026-05-29

- ✓ 整理 `docs/` 文档结构 (架构 / 功能开发 / 实验评测 / 项目管理)
- ✓ 把 codex skill 接入 nanobot builtin
- ✓ 增加 skill selection trace + 10 任务统一 eval (`run_eval.py`、`./eval.sh`)
- ✓ 写参考 spreadsheet skills 分析 (`reference-spreadsheet-skills.md`)
- ✓ 写 mentor demo 路线第一版 (市州表 + 5 个 tc-* skill) → 弃用, 因任务太简单
- ✓ 第二版 mentor demo (欠费表 + 2 个 tc-bigtable-* skill), 跑通双轨迹对照
- ✓ 拆分 `eval_test/results/` 为 `mentor_demo/` 和 `skill_matrix/` 两条独立线
- ✓ 拆分 `docs/实验评测/` 为 `mentor-demo/` 和 `skill-matrix/` 两个子目录, 新增索引 README
- ✓ `run_demo.py` 加时间戳归档 (`runs/<YYYYMMDD-HHMMSS>/`)
- ✓ 删除 `trace_skill_selection_matrix.py` wrapper、加 `.gitignore`
- ✓ 清空 workspace 运行态 (保留 USER/SOUL/AGENTS/HEARTBEAT/MEMORY 模板)
- ✓ 同步 docs/README、docs/架构、docs/功能开发、docs/项目管理 全部到新结构
- ✓ 修正定位: TableClaw 是**表格专精 agent**, 覆盖 QA + 编辑 + 转换 + 报表 4 大场景
- ✓ 建立 TODO + dev-log 双文档分工

维护规则

- 新 TODO 加在对应时间段（近期 / 中期 / 长期）下、合适的主题分组里。
- 完成一项：复选框打勾、日期写完成日、移到"已完成"段落。
- 计划变更：删除 TODO 时在 dev-log 里说明原因，避免 TODO 静悄悄消失。
- 时间段不是死的：近期可以跨周，但每周扫一眼"近 7 天进度"小节做节奏校准。